

Retail Data Analysis: Sales Analysis and Customer Segmentation

11-19-2024

1. Load Data

```
# Read the Excel file
df <- read_excel("C:/Users/Wenwen/Desktop/case study/online_retail_II.xlsx")
head(df) # Display the first few rows of the dataset
```

```
## # A tibble: 6 x 8
##   Invoice StockCode Description Quantity InvoiceDate      Price 'Customer ID'
##   <chr>   <chr>      <chr>      <dbl> <dtm>      <dbl>      <dbl>
## 1 489434  85048      "15CM CHRI~    12 2009-12-01 07:45:00  6.95      13085
## 2 489434  79323P      "PINK CHER~    12 2009-12-01 07:45:00  6.75      13085
## 3 489434  79323W      "WHITE CHE~    12 2009-12-01 07:45:00  6.75      13085
## 4 489434  22041      "RECORD FR~    48 2009-12-01 07:45:00  2.1       13085
## 5 489434  21232      "STRAWBERR~    24 2009-12-01 07:45:00  1.25      13085
## 6 489434  22064      "PINK DOUG~    24 2009-12-01 07:45:00  1.65      13085
## # i 1 more variable: Country <chr>
```

2. Data Discovery Summary Statistics

```
summary(df)
```

```
##      Invoice      StockCode      Description      Quantity
## Length:525461 Length:525461 Length:525461 Min. : -9600.00
## Class :character Class :character Class :character 1st Qu.: 1.00
## Mode :character Mode :character Mode :character Median : 3.00
##                                     Mean : 10.34
##                                     3rd Qu.: 10.00
##                                     Max. :19152.00
##
##      InvoiceDate      Price      Customer ID
## Min. :2009-12-01 07:45:00.00 Min. : -53594.36 Min. :12346
## 1st Qu.:2010-03-21 12:20:00.00 1st Qu.: 1.25 1st Qu.:13983
## Median :2010-07-06 09:51:00.00 Median : 2.10 Median :15311
## Mean :2010-06-28 11:37:36.84 Mean : 4.69 Mean :15361
## 3rd Qu.:2010-10-15 12:45:00.00 3rd Qu.: 4.21 3rd Qu.:16799
## Max. :2010-12-09 20:01:00.00 Max. : 25111.09 Max. :18287
##                                     NA's :107927
##
##      Country
## Length:525461
## Class :character
## Mode :character
##
##
##
```

Data Structure

```
str(df)
```

```
## tibble [525,461 x 8] (S3: tbl_df/tbl/data.frame)
## $ Invoice      : chr [1:525461] "489434" "489434" "489434" "489434" ...
## $ StockCode    : chr [1:525461] "85048" "79323P" "79323W" "22041" ...
## $ Description: chr [1:525461] "15CM CHRISTMAS GLASS BALL 20 LIGHTS" "PINK CHERRY LIGHTS" "WHITE CHE
## $ Quantity     : num [1:525461] 12 12 12 48 24 24 24 10 12 12 ...
## $ InvoiceDate: POSIXct[1:525461], format: "2009-12-01 07:45:00" "2009-12-01 07:45:00" ...
## $ Price        : num [1:525461] 6.95 6.75 6.75 2.1 1.25 1.65 1.25 5.95 2.55 3.75 ...
## $ Customer ID: num [1:525461] 13085 13085 13085 13085 13085 ...
## $ Country      : chr [1:525461] "United Kingdom" "United Kingdom" "United Kingdom" "United Kingdom" .
```

Unique Products

```
n_distinct(df$Description)
```

```
## [1] 4644
```

Missing Values

```
colSums(is.na(df))
```

```
##      Invoice      StockCode Description      Quantity InvoiceDate      Price
##           0           0         2928           0           0           0
## Customer ID      Country
##      107927           0
```

Unique Customers

3. Data Cleaning

Remove Duplicates

```
sum(duplicated(df))
```

```
## [1] 6865
```

```
df_no_duplicates <- df %>% distinct()
cat("Number of rows after removing duplicates:", nrow(df_no_duplicates), "\n")
```

```
## Number of rows after removing duplicates: 518596
```

```
cat("Number of rows before removing duplicates:", nrow(df), "\n")
```

```
## Number of rows before removing duplicates: 525461
```

Handle Missing Values

```
colSums(is.na(df_no_duplicates))
```

```
##      Invoice  StockCode Description  Quantity InvoiceDate      Price
##          0         0         2928          0           0          0
## Customer ID      Country
##    107833          0
```

Fill Missing Descriptions

```
# Function to get the mode of a vector
get_mode <- function(x) {
  ux <- unique(x)
  ux[which.max(tabulate(match(x, ux)))]
}

# Fill missing descriptions based on the mode of the StockCode
df_no_duplicates <- df_no_duplicates %>%
  group_by(StockCode) %>%
  mutate(Description = ifelse(is.na(Description), get_mode(Description), Description)) %>%
  ungroup()

# Replace any remaining NA descriptions with "No Description"
df_no_duplicates <- df_no_duplicates %>%
  mutate(Description = replace_na(Description, "No Description"))

# Check for missing values again
colSums(is.na(df_no_duplicates))
```

```
##      Invoice  StockCode Description  Quantity InvoiceDate      Price
##          0         0           0          0           0          0
## Customer ID      Country
##    107833          0
```

Handle Missing Customer IDs

```
# Replace missing Customer IDs with '-1' for type conversion
df_no_duplicates <- df_no_duplicates %>%
  mutate(`Customer ID` = ifelse(is.na(`Customer ID`), -1, `Customer ID`))

head(df_no_duplicates)
```

```
## # A tibble: 6 x 8
##   Invoice StockCode Description Quantity InvoiceDate      Price `Customer ID`
##   <chr>   <chr>    <chr>      <dbl> <dtm>      <dbl>      <dbl>
## 1 489434  85048    "15CM CHRI~    12 2009-12-01 07:45:00  6.95      13085
## 2 489434  79323P    "PINK CHER~    12 2009-12-01 07:45:00  6.75      13085
## 3 489434  79323W    "WHITE CHE~    12 2009-12-01 07:45:00  6.75      13085
## 4 489434  22041    "RECORD FR~    48 2009-12-01 07:45:00  2.1       13085
## 5 489434  21232    "STRAWBERR~    24 2009-12-01 07:45:00  1.25      13085
## 6 489434  22064    "PINK DOUG~    24 2009-12-01 07:45:00  1.65      13085
## # i 1 more variable: Country <chr>
```

For null values in CustomerID, we replace them with '-1' to enable type conversion, as the current type cannot handle null values.

For the Description column, we fill missing values by mapping the StockCode to its corresponding description, and set 'No Description' for any remaining null values.

4. Data Manipulation Extract Time-Based Features

```
# Extract month, week, and day from InvoiceDate
df_no_duplicates <- df_no_duplicates %>%
  mutate(
    Month = month(InvoiceDate),
    MonthText = month(InvoiceDate, label = TRUE, abbr = TRUE),
    Week = week(InvoiceDate),
    Day = day(InvoiceDate)
  )

head(df_no_duplicates)
```

```
## # A tibble: 6 x 12
##   Invoice StockCode Description Quantity InvoiceDate      Price 'Customer ID'
##   <chr>   <chr>      <chr>      <dbl> <dtm>      <dbl>      <dbl>
## 1 489434  85048      "15CM CHRI~      12 2009-12-01 07:45:00  6.95      13085
## 2 489434  79323P      "PINK CHER~      12 2009-12-01 07:45:00  6.75      13085
## 3 489434  79323W      "WHITE CHE~      12 2009-12-01 07:45:00  6.75      13085
## 4 489434  22041      "RECORD FR~      48 2009-12-01 07:45:00  2.1       13085
## 5 489434  21232      "STRAWBERR~      24 2009-12-01 07:45:00  1.25      13085
## 6 489434  22064      "PINK DOUG~      24 2009-12-01 07:45:00  1.65      13085
## # i 5 more variables: Country <chr>, Month <dbl>, MonthText <ord>, Week <dbl>,
## #   Day <int>
```

5. Exploratory Data Analysis (EDA) Summary of Cleaned Data

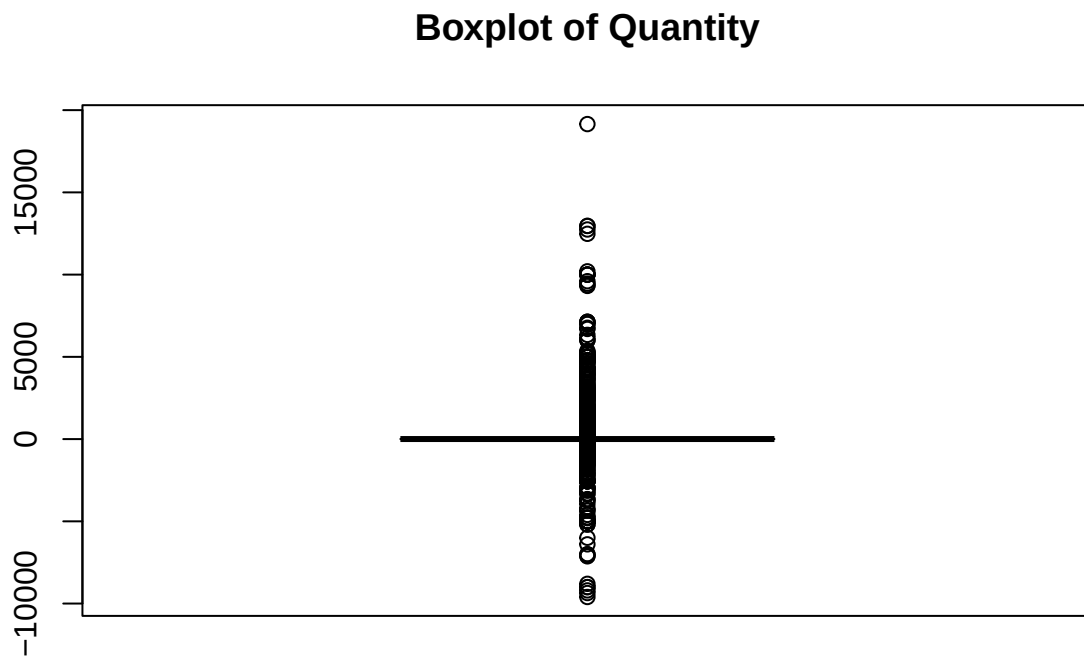
```
summary(df_no_duplicates)
```

```
##      Invoice      StockCode      Description      Quantity
## Length:518596 Length:518596 Length:518596 Min.   :-9600.00
## Class :character Class :character Class :character 1st Qu.:  1.00
## Mode  :character Mode  :character Mode  :character Median  :   3.00
##                                     Mean   :  10.44
##                                     3rd Qu.:  10.00
##                                     Max.   :19152.00
##
##      InvoiceDate      Price      Customer ID
## Min.   :2009-12-01 07:45:00.00 Min.   :-53594.36 Min.   :  -1
## 1st Qu.:2010-03-21 11:07:00.00 1st Qu.:   1.25 1st Qu.:12747
## Median :2010-07-05 15:42:00.00 Median :   2.10 Median :14606
## Mean   :2010-06-28 06:43:01.56 Mean   :   4.72 Mean  :12161
## 3rd Qu.:2010-10-15 11:40:00.00 3rd Qu.:   4.21 3rd Qu.:16441
## Max.   :2010-12-09 20:01:00.00 Max.   :25111.09 Max.   :18287
##
##      Country      Month      MonthText      Week
## Length:518596 Min.   : 1.000 Nov    : 76584 Min.   : 1.00
```

```
## Class :character 1st Qu.: 4.000 Dec : 66924 1st Qu.:17.00
## Mode :character Median : 8.000 Oct : 58249 Median :33.00
## Mean : 7.434 Sep : 41607 Mean :30.38
## 3rd Qu.:11.000 Mar : 40985 3rd Qu.:45.00
## Max. :12.000 Jun : 39469 Max. :51.00
## (Other):194778
##
## Day
## Min. : 1.00
## 1st Qu.: 8.00
## Median :15.00
## Mean :15.25
## 3rd Qu.:23.00
## Max. :31.00
##
```

Boxplot of Quantity

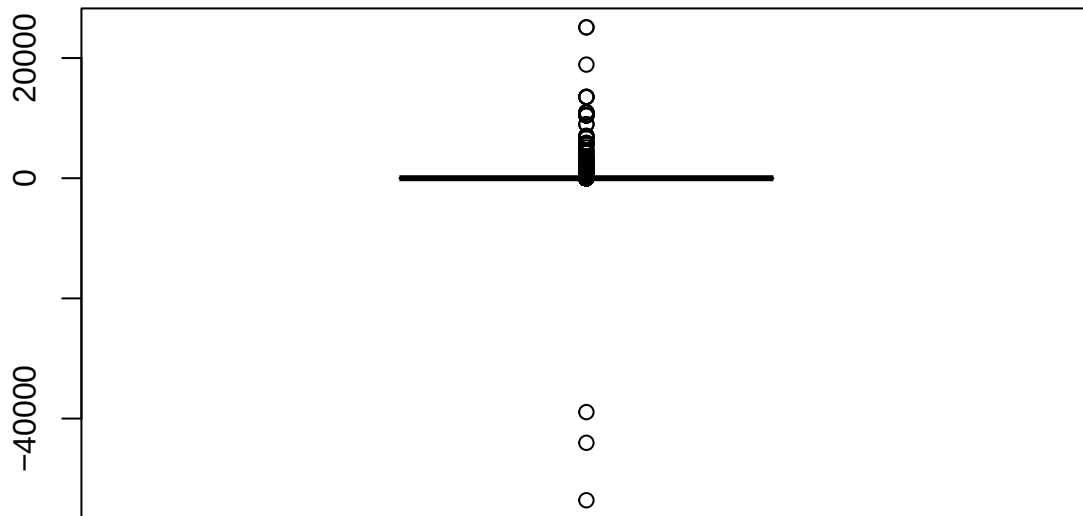
```
boxplot(df_no_duplicates$Quantity, main = "Boxplot of Quantity")
```



Boxplot of Price

```
boxplot(df_no_duplicates$Price, main = "Boxplot of Price")
```

Boxplot of Price



5.2 Sales Trend by Time

Calculate Sales

```
# Calculate Sales as the product of Quantity and Price
df_no_duplicates <- df_no_duplicates %>%
  mutate(Sales = Quantity * Price)
```

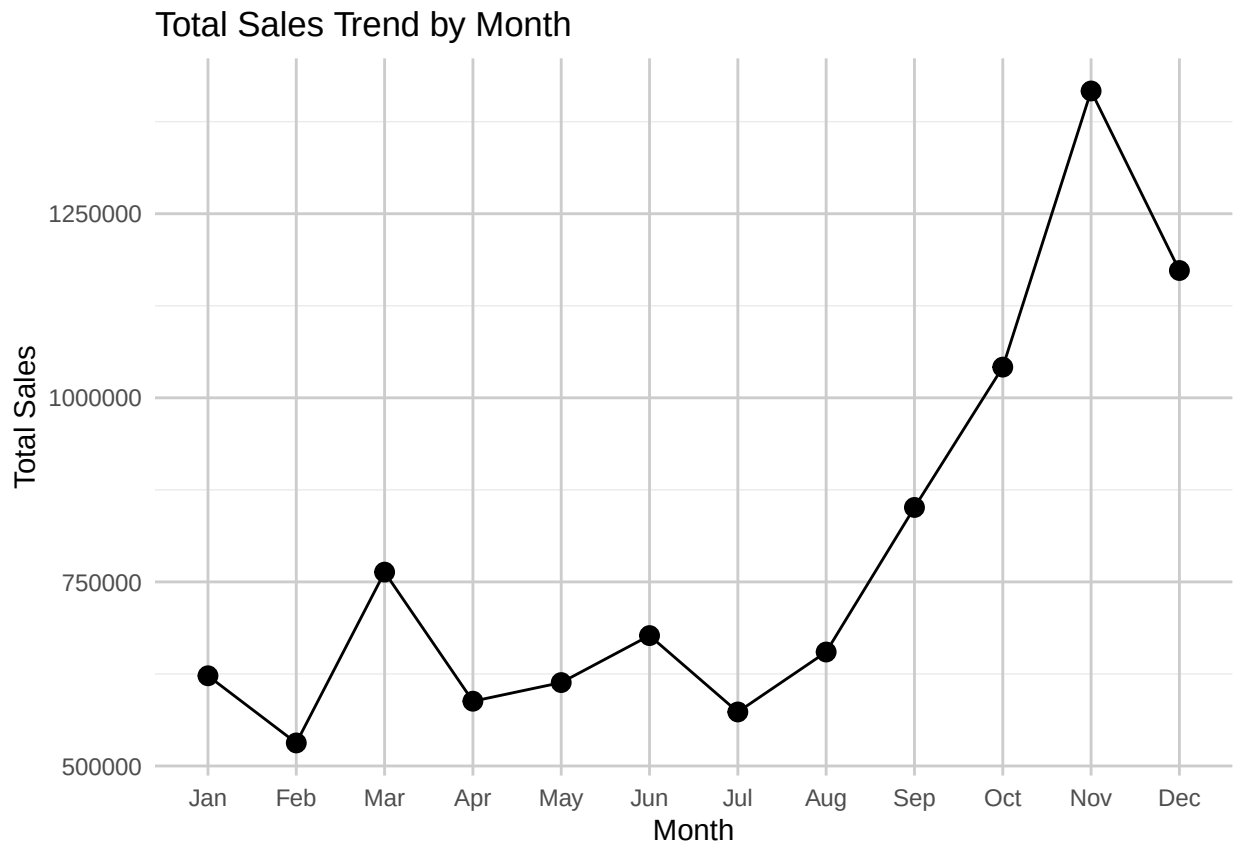
Monthly Sales Trend

```
# Group by Month and calculate total sales
monthly_sales <- df_no_duplicates %>%
  group_by(Month) %>%
  summarize(TotalSales = sum(Sales))

# Define month names for plotting
months <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
            "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")

# Plot the total sales trend by month
ggplot(monthly_sales, aes(x = factor(Month, labels = months), y = TotalSales)) +
  geom_point(size = 3) + # Use points to represent sales
  geom_line(group = 1) + # Connect points with lines
  labs(title = "Total Sales Trend by Month",
       x = "Month",
       y = "Total Sales") +
```

```
theme_minimal() +
theme(panel.grid.major = element_line(color = "gray80"))
```

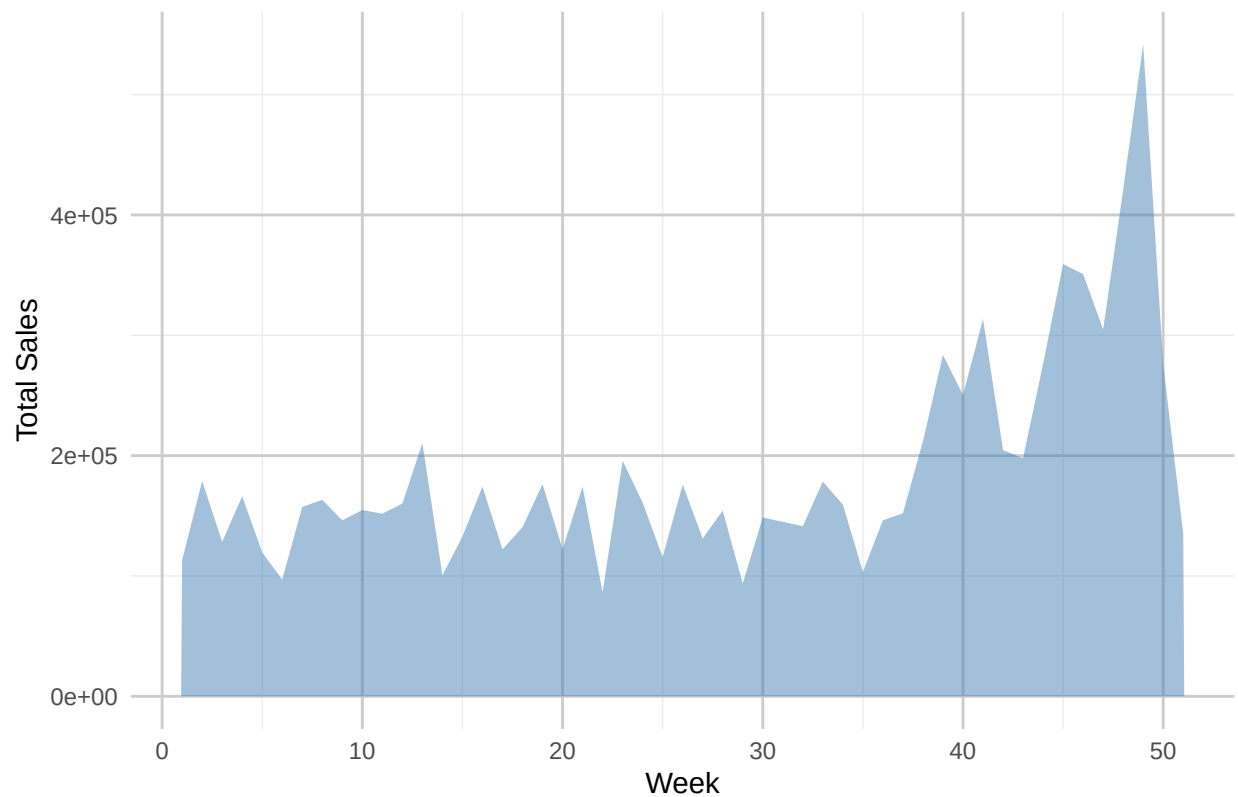


Weekly Sales Trend

```
# Group by Week and calculate total sales
weekly_sales <- df_no_duplicates %>%
  group_by(Week) %>%
  summarize(TotalSales = sum(Sales))

# Plot the sales trend by week using an area plot
ggplot(weekly_sales, aes(x = Week, y = TotalSales)) +
  geom_area(fill = "steelblue", alpha = 0.5) + # Create an area plot
  labs(title = "Sales Trend by Week",
       x = "Week",
       y = "Total Sales") +
  theme_minimal() +
  theme(panel.grid.major = element_line(color = "gray80"))
```

Sales Trend by Week



Daily Sales Trend

```
# Extract day of the week from InvoiceDate
df_no_duplicates <- df_no_duplicates %>%
  mutate(DayOfWeek = wday(InvoiceDate, label = TRUE, abbr = TRUE))

# Group by DayOfWeek and calculate total sales
day_of_week_sales <- df_no_duplicates %>%
  group_by(DayOfWeek) %>%
  summarize(TotalSales = sum(Sales))

# Plot the total sales trend by day of the week
ggplot(day_of_week_sales, aes(x = DayOfWeek, y = TotalSales)) +
  geom_point(size = 3) + # Use points to represent sales
  geom_line(group = 1) + # Connect points with lines
  labs(title = "Total Sales Trend by Day of Week",
       x = "Day of Week",
       y = "Total Sales") +
  theme_minimal() +
  theme(panel.grid.major = element_line(color = "gray80"))
```

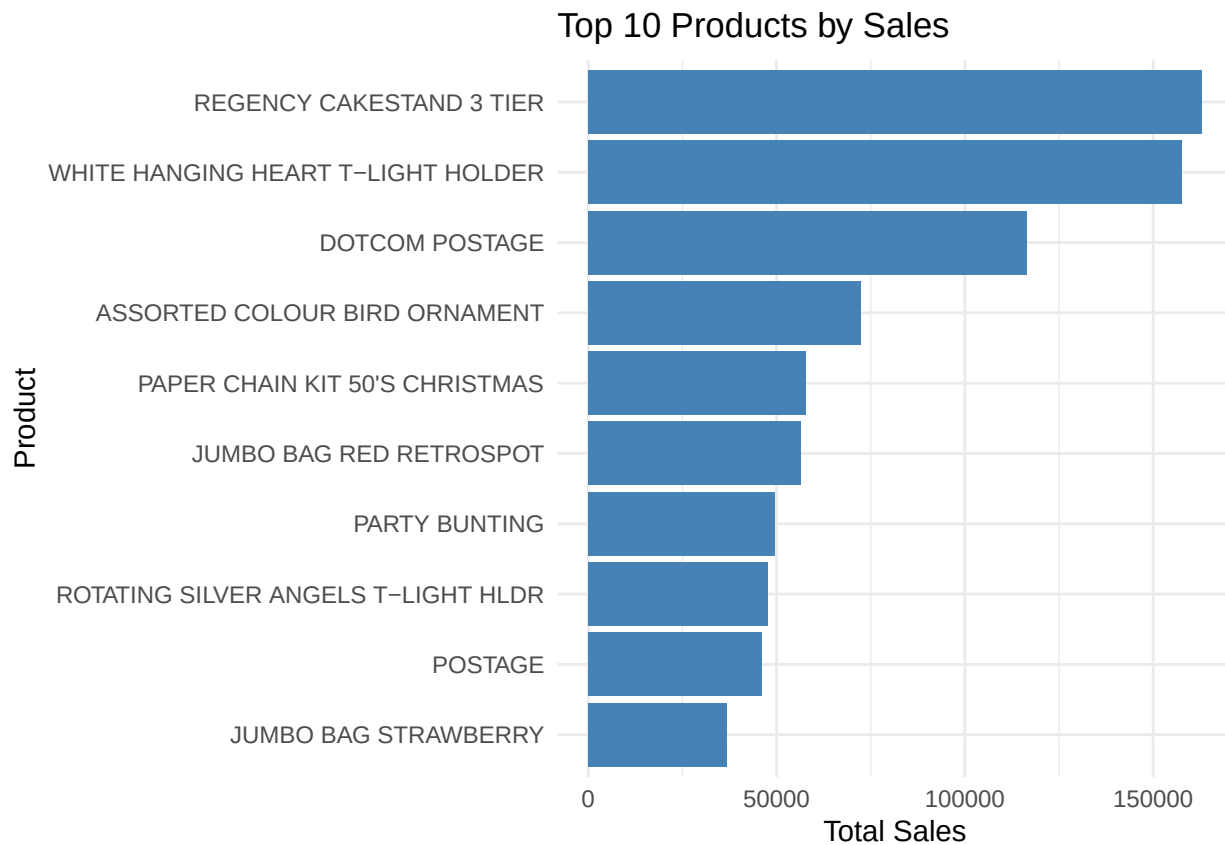



The highest sales occur between September and December, peaking in November. The week with the largest sales is week 49, just before the Christmas holiday. Customers make the fewest purchases on weekends, with almost no sales on Saturdays.

5.3 Sales by Product Top 10 Products by Sales

```
# Group by Description, calculate total sales, and get the top 10 products
product_sales <- df_no_duplicates %>%
  group_by(Description) %>%
  summarize(TotalSales = sum(Sales)) %>%
  arrange(desc(TotalSales)) %>%
  head(10)

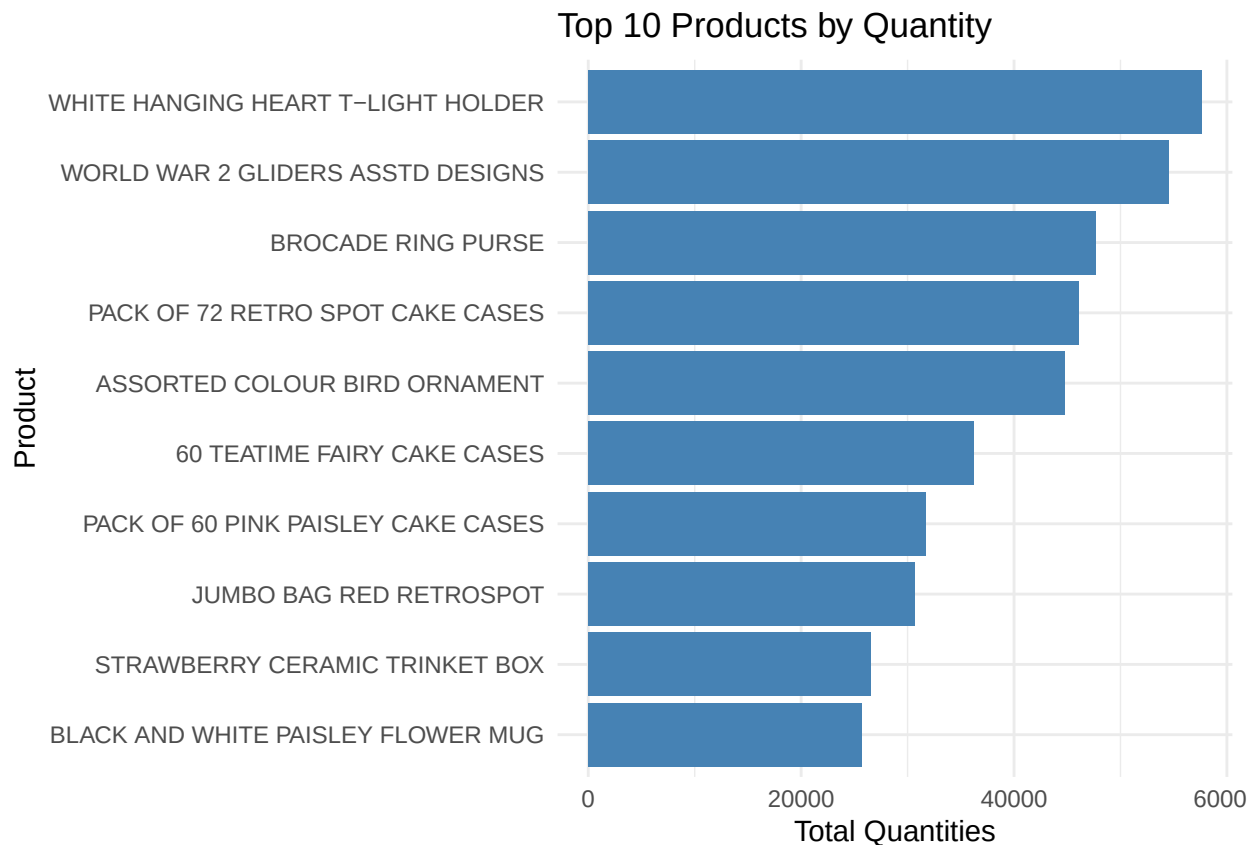
# Plot the top 10 products by sales
ggplot(product_sales, aes(x = TotalSales, y = reorder(Description, TotalSales))) +
  geom_bar(stat = "identity", fill = "steelblue") + # Create a horizontal bar plot
  labs(title = "Top 10 Products by Sales",
       x = "Total Sales",
       y = "Product") +
  theme_minimal()
```



Top 10 Products by Quantity

```
# Group by Description, calculate total quantities, and get the top 10 products
product_quantities <- df_no_duplicates %>%
  group_by(Description) %>%
  summarize(TotalQuantity = sum(Quantity)) %>%
  arrange(desc(TotalQuantity)) %>%
  head(10)

# Plot the top 10 products by quantities
ggplot(product_quantities, aes(x = TotalQuantity, y = reorder(Description, TotalQuantity))) +
  geom_bar(stat = "identity", fill = "steelblue") + # Create a horizontal bar plot
  labs(title = "Top 10 Products by Quantity",
       x = "Total Quantities",
       y = "Product") +
  theme_minimal()
```



Overlapping Products

```
# Merge product_sales and product_quantities on Description
product_sales_quantity <- merge(
  product_sales,
  product_quantities,
  by = "Description",
  suffixes = c("_sales", "_quantity")
)

# Calculate the percentage of total sales accounted for by the top 10 products
total_sales <- sum(df_no_duplicates$Sales)
top_10_sales <- sum(product_sales$TotalSales)
percentage_top_10_sales <- (top_10_sales / total_sales) * 100
percentage_top_10_sales
```

```
## [1] 8.450512
```

5.4 Sales by Country

Sales by Country

```
# Group by Country and calculate total sales
sales_by_country <- df_no_duplicates %>%
  group_by(Country) %>%
  summarize(TotalSales = sum(Sales)) %>%
```

```

arrange(desc(TotalSales))

# Summary of sales by country
summary(sales_by_country$TotalSales)

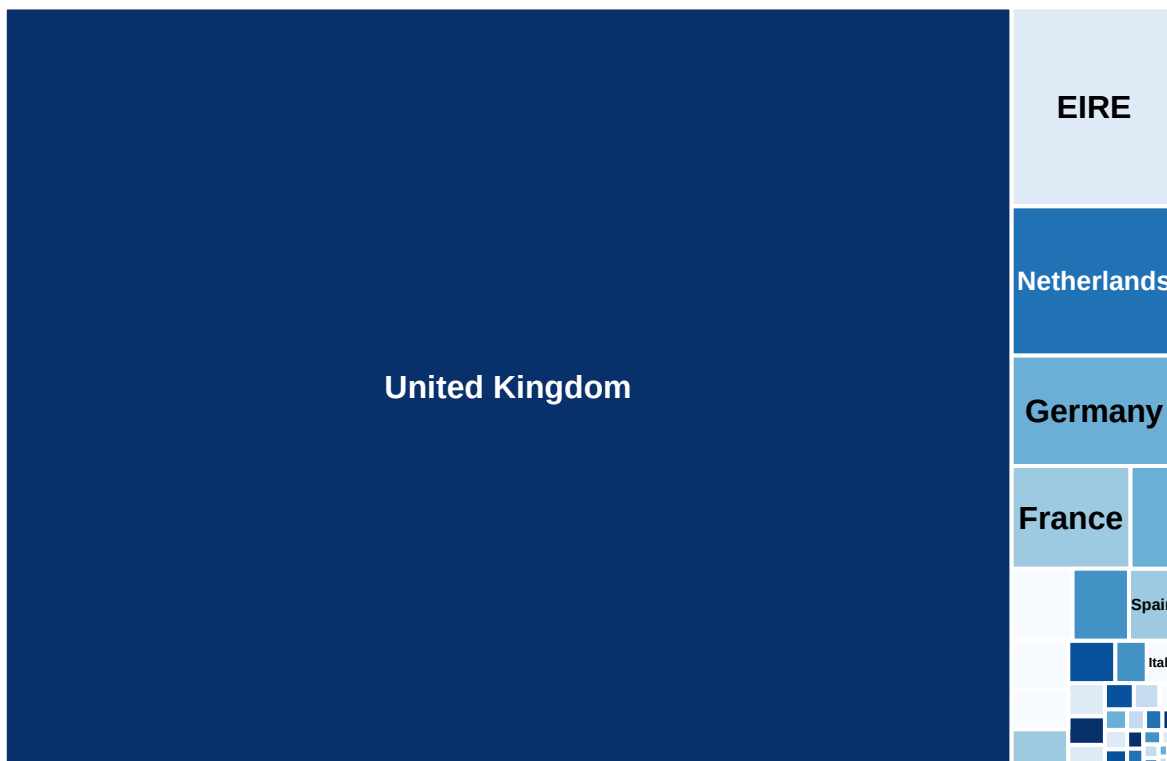
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      -7    2607    6063  237644   25337 8161693

# Filter out negative sales
sales_by_country <- sales_by_country %>%
  filter(TotalSales >= 0)

# Treemap of sales by country
treemap(sales_by_country,
  index = "Country",
  vSize = "TotalSales",
  type = "index",
  title = "Sales by Country (Treemap)",
  fontsize.title = 14,
  fontsize.labels = 12,
  palette = "Blues",
  border.col = "white",
  alpha = 0.8)

```

Sales by Country (Treemap)



Top 5 Countries

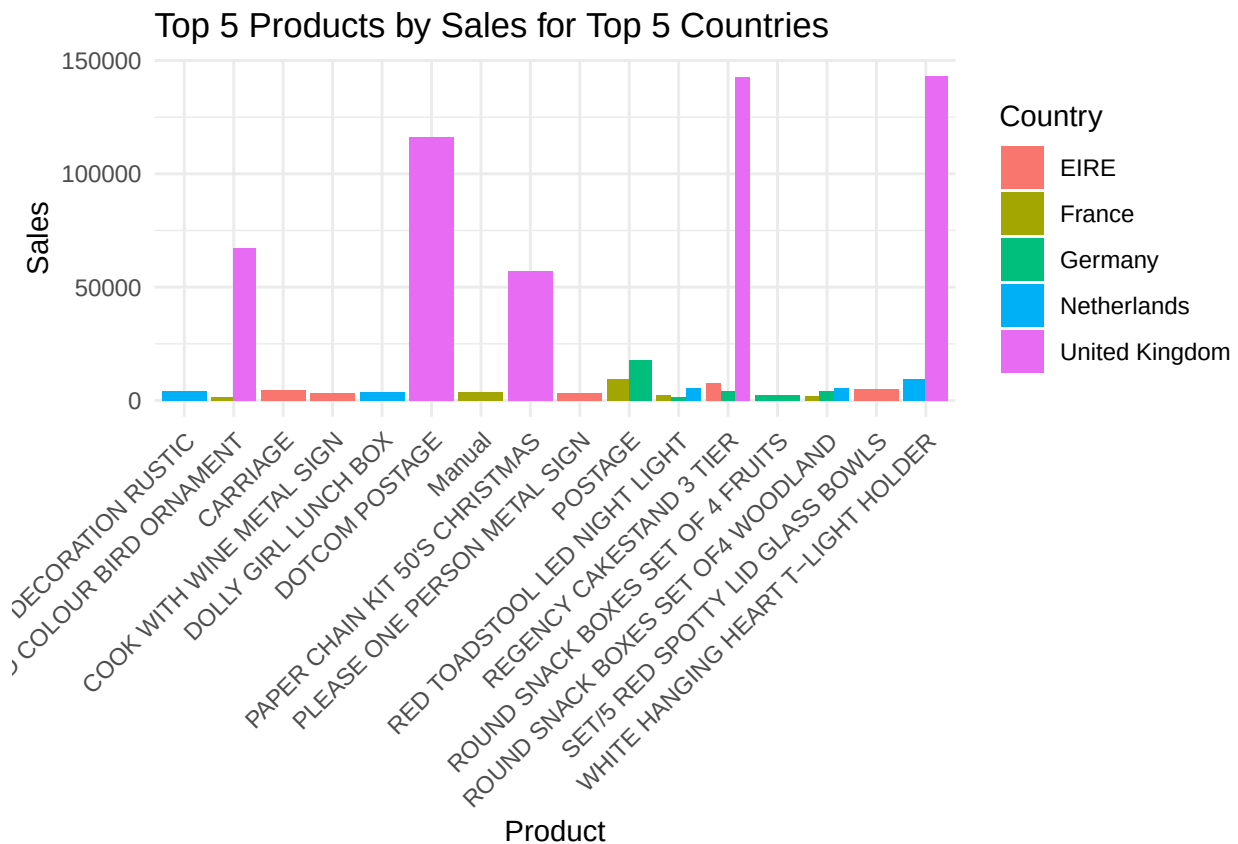
```

# Get the top 5 countries by sales
top_5_countries <- sales_by_country %>%
  head(5) %>%
  pull(Country)

# Top products by sales for the top 5 countries
top_products_by_country <- df_no_duplicates %>%
  filter(Country %in% top_5_countries) %>%
  group_by(Country, Description) %>%
  summarize(TotalSales = sum(Sales), .groups = 'drop') %>%
  group_by(Country) %>%
  group_modify(~ .x %>% arrange(desc(TotalSales))) %>%
  slice_head(n = 5)

# Plot the top 5 products by sales for the top 5 countries
ggplot(top_products_by_country, aes(x = Description, y = TotalSales, fill = Country)) +
  geom_bar(stat = "identity", position = "dodge") + # Create grouped bar plot
  labs(title = "Top 5 Products by Sales for Top 5 Countries",
       x = "Product",
       y = "Sales") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

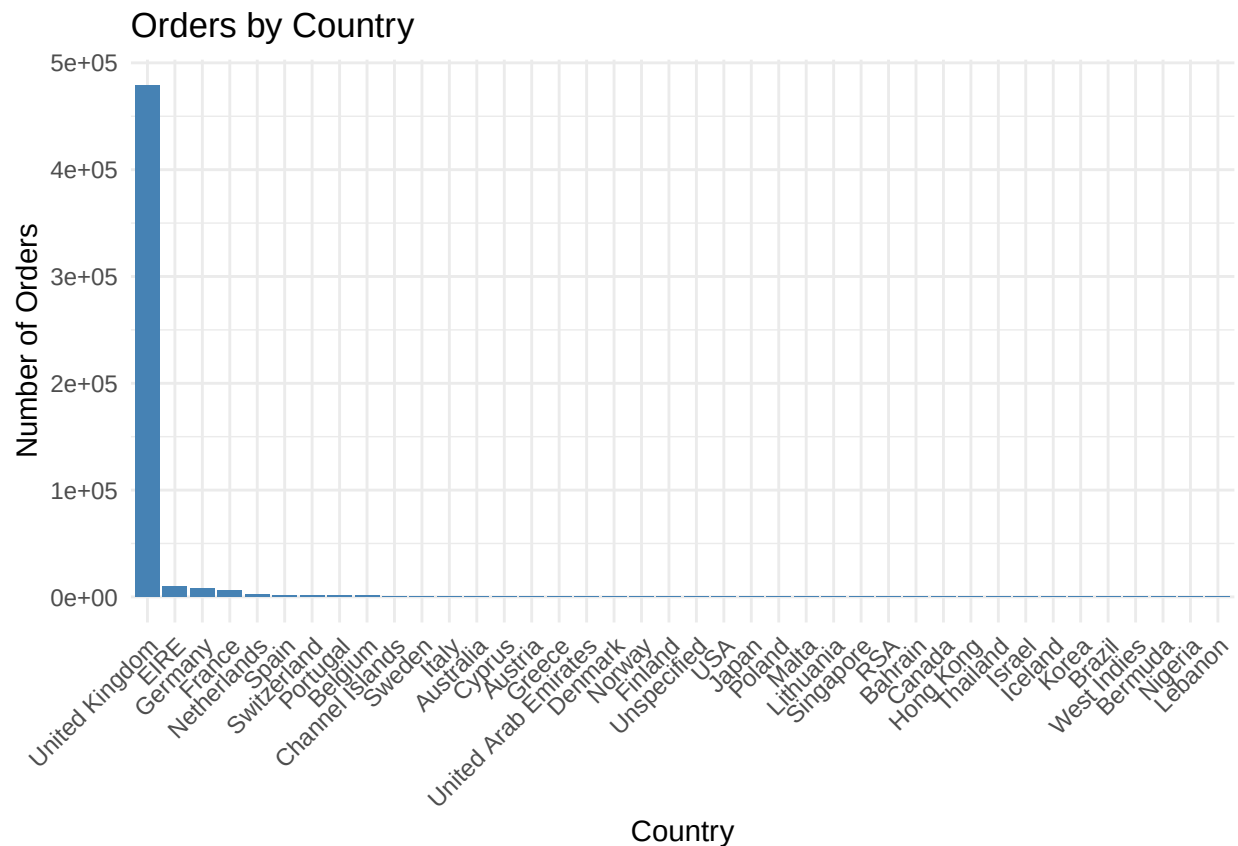
```



Orders by Country

```
# Count the number of orders by country
orders_by_country <- df_no_duplicates %>%
  count(Country, sort = TRUE)

# Plot the number of orders by country
ggplot(orders_by_country, aes(x = reorder(Country, -n), y = n)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  labs(title = "Orders by Country",
       x = "Country",
       y = "Number of Orders") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The UK accounts for the largest portion of sales, followed by EIRE, the Netherlands, Germany, and France. Each country has a distinct set of top products by sales, highlighting the potential for personalized marketing. Orders are positively correlated with sales.

5.5 Customer Behavior Rename Customer ID Column

```
df_no_duplicates <- df_no_duplicates %>%
  rename(CustomerID = `Customer ID`)
```

Top 10 Customers by Sales

```

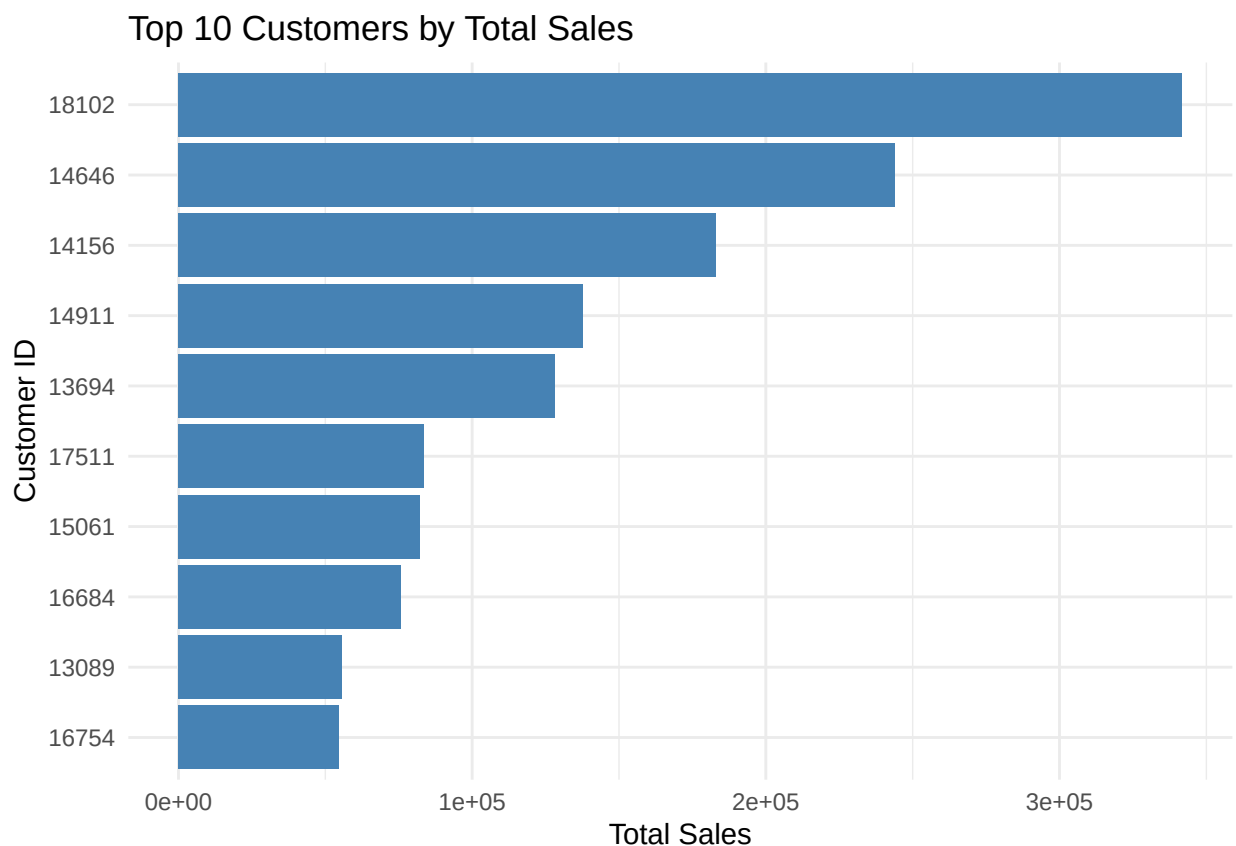
# Group by CustomerID and calculate total sales
top_10_customers <- df_no_duplicates %>%
  group_by(CustomerID) %>%
  summarize(TotalSales = sum(Sales)) %>%
  arrange(desc(TotalSales)) %>%
  head(11) %>%
  filter(CustomerID != -1)

# Convert CustomerID to character
top_10_customers <- top_10_customers %>%
  mutate(CustomerID = as.character(CustomerID))

# Filter out CustomerID == "-1" and sort by TotalSales
top_10_customers_filtered <- top_10_customers %>%
  filter(CustomerID != "-1") %>%
  arrange(desc(TotalSales))

# Plot the top 10 customers by total sales
ggplot(top_10_customers_filtered, aes(x = TotalSales, y = reorder(CustomerID, TotalSales))) +
  geom_bar(stat = "identity", fill = "steelblue") + # Create horizontal bar plot
  labs(title = "Top 10 Customers by Total Sales",
       x = "Total Sales",
       y = "Customer ID") +
  theme_minimal()

```



Top 10 Customers by Sales and Quantities

```

# Group by CustomerID, calculate total sales and quantities
top_10_sales_customers <- df_no_duplicates %>%
  group_by(CustomerID) %>%
  summarize(TotalSales = sum(Sales),
             TotalQuantity = sum(Quantity)) %>%
  arrange(desc(TotalSales)) %>%
  head(10) %>%
  filter(CustomerID != -1)

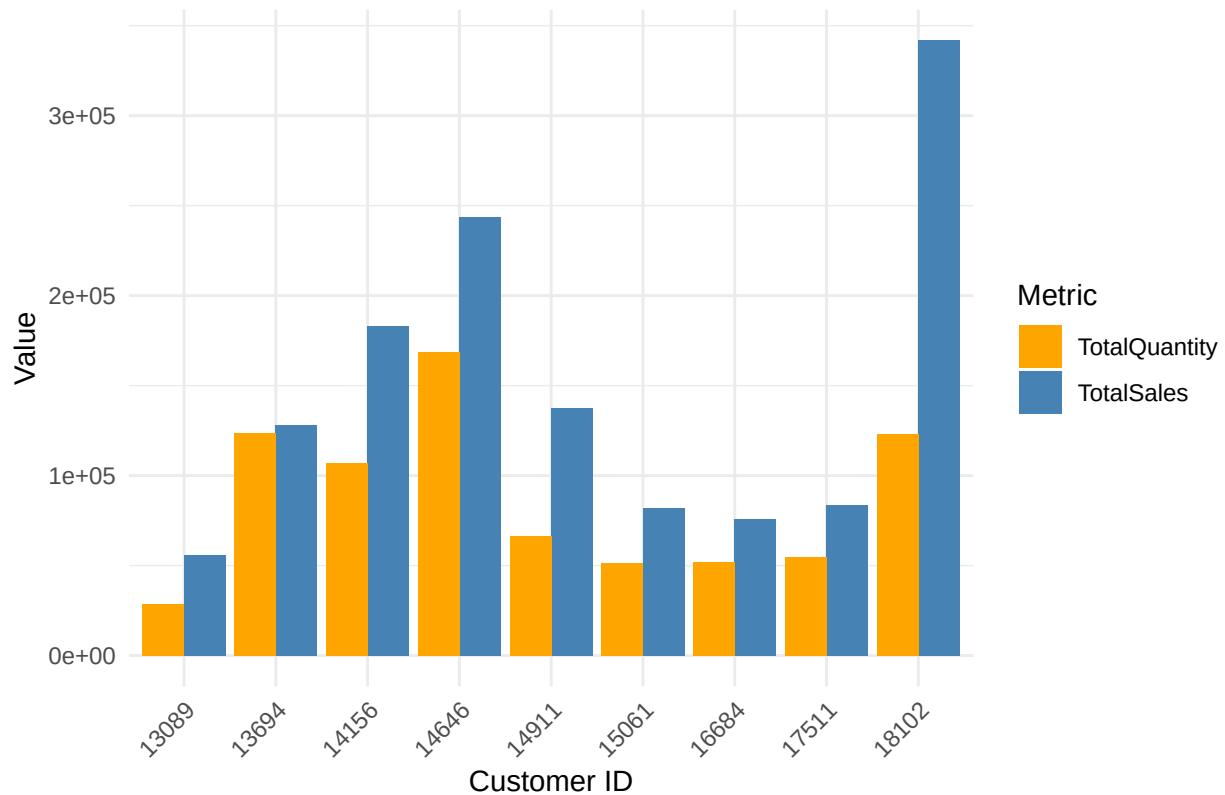
# Convert CustomerID to character
top_10_sales_customers <- top_10_sales_customers %>%
  mutate(CustomerID = as.character(CustomerID))

# Reshape the data for plotting
top_10_sales_customers_long <- top_10_sales_customers %>%
  pivot_longer(cols = c(TotalSales, TotalQuantity), names_to = "Metric", values_to = "Value")

# Plot the top 10 customers by sales and quantities
ggplot(top_10_sales_customers_long, aes(x = CustomerID, y = Value, fill = Metric)) +
  geom_bar(stat = "identity", position = "dodge") + # Create grouped bar plot
  labs(title = "Top 10 Customers by Sales and Quantities",
       x = "Customer ID",
       y = "Value") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) + # Rotate x-axis labels
  scale_fill_manual(values = c("TotalSales" = "steelblue", "TotalQuantity" = "orange")) + # Custom col
  guides(fill = guide_legend(title = "Metric")) # Add legend

```


Top 10 Customers by Sales and Quantities



Customer Sales Over Time

```
# Extract YearMonth from InvoiceDate
df_no_duplicates$YearMonth <- format(df_no_duplicates$InvoiceDate, "%Y-%m")

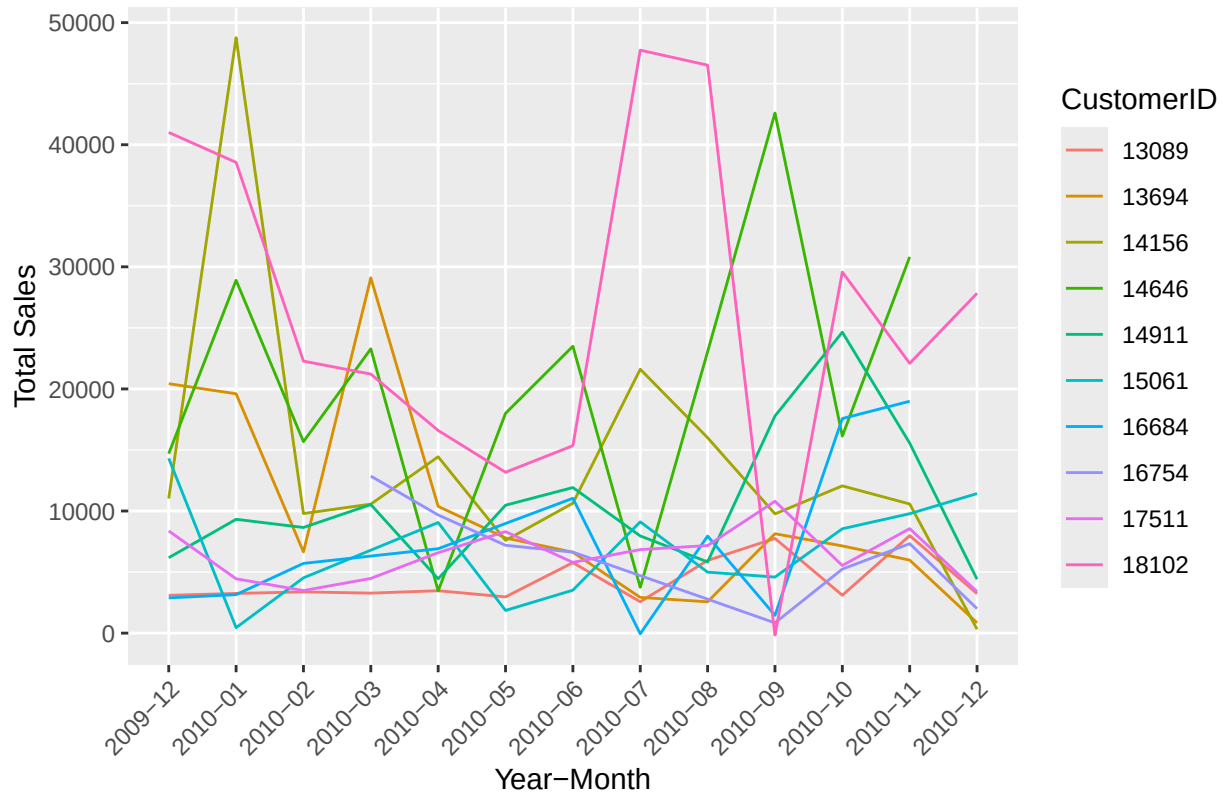
# Filter data for top 10 customers
df_no_duplicates_top_10_customers <- df_no_duplicates[df_no_duplicates$CustomerID %in% top_10_customers$CustomerID, ]

# Group by CustomerID and YearMonth, calculate total sales
customer_sales_over_time <- df_no_duplicates_top_10_customers %>%
  group_by(CustomerID, YearMonth) %>%
  summarise(Sales = sum(Sales), .groups = 'drop')

# Convert YearMonth and CustomerID to character
customer_sales_over_time$YearMonth <- as.character(customer_sales_over_time$YearMonth)
customer_sales_over_time$CustomerID <- as.character(customer_sales_over_time$CustomerID)

# Plot customer sales over time
ggplot(customer_sales_over_time[customer_sales_over_time$CustomerID != "-1", ], aes(x = YearMonth, y = Sales)) +
  geom_line() +
  labs(title = "Customer Sales Over Time", x = "Year-Month", y = "Total Sales") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Customer Sales Over Time



Top 10 Customers Summary

```
top_10_customers <- df_no_duplicates_top_10_customers[df_no_duplicates_top_10_customers$CustomerID != -1]
head(top_10_customers)
```

```
## # A tibble: 6 x 15
##   Invoice StockCode Description      Quantity InvoiceDate      Price CustomerID
##   <chr>   <chr>      <chr>          <dbl> <dtm>          <dbl>      <dbl>
## 1 489438  21329    DINOSAURS WR~         28 2009-12-01 09:24:00  0.98      18102
## 2 489438  21252    SET OF MEADOW~         30 2009-12-01 09:24:00  1.69      18102
## 3 489438  21100    CHARLIE AND L~         30 2009-12-01 09:24:00  1.15      18102
## 4 489438  21033    JUMBO BAG CHA~         30 2009-12-01 09:24:00  2         18102
## 5 489438  20711    JUMBO BAG TOYS         60 2009-12-01 09:24:00  1.3       18102
## 6 489438  21410    COUNTRY COTTA~         32 2009-12-01 09:24:00  2.5       18102
## # i 8 more variables: Country <chr>, Month <dbl>, MonthText <ord>, Week <dbl>,
## #   Day <int>, Sales <dbl>, DayOfWeek <ord>, YearMonth <chr>
```

```
# Summarize top 10 customers by sales, purchase frequency, and country
top_10_customers_summary <- top_10_customers %>%
  group_by(CustomerID) %>%
  summarise(Sales = sum(Sales), PurchaseFrequency = n(), .groups = 'drop') %>%
  arrange(desc(Sales))
```

```
# Get the most frequent country for each customer
top_10_customers_country <- top_10_customers %>%
```

```

group_by(CustomerID) %>%
  summarise(Country = names(sort(table(Country), decreasing = TRUE))[1], .groups = 'drop')

# Merge summaries
top_10_customers_summary <- merge(top_10_customers_summary, top_10_customers_country, by = "CustomerID")
colnames(top_10_customers_summary)[colnames(top_10_customers_summary) == "Invoice"] <- "PurchaseFrequency"
top_10_customers_summary

```

##	CustomerID	Sales	PurchaseFrequency	Country
## 1	13089	55801.73	1579	United Kingdom
## 2	13694	128172.42	973	United Kingdom
## 3	14156	183163.55	2708	EIRE
## 4	14646	243853.05	1805	Netherlands
## 5	14911	137710.76	5707	EIRE
## 6	15061	82162.74	588	United Kingdom
## 7	16684	75610.17	468	United Kingdom
## 8	16754	54558.18	165	United Kingdom
## 9	17511	83760.60	1058	United Kingdom
## 10	18102	341776.73	635	United Kingdom

We identified the top 10 customers, and found a positive correlation between their purchasing power and quantities. Throughout the year, the top customers do not exhibit any clear seasonal purchasing patterns. Eight of the top 10 customers are from the UK, with the second-largest customer hailing from the Netherlands.

6. Customer Segmentation - Clustering

Calculate Recency

```

recent_date <- max(df_no_duplicates$InvoiceDate)
df_no_duplicates$Recency <- as.numeric(difftime(recent_date, df_no_duplicates$InvoiceDate, units = "days"))

```

RFM Analysis

```

# Calculate Recency, Frequency, and MonetaryValue for each customer
customer_rfm <- df_no_duplicates %>%
  group_by(CustomerID) %>%
  summarise(
    Recency = as.numeric(difftime(recent_date, max(InvoiceDate), units = "days")),
    Frequency = n_distinct(Invoice),
    MonetaryValue = sum(Sales),
    .groups = 'drop'
  ) %>%
  filter(CustomerID != -1)

# Calculate quantiles for Recency, Frequency, and MonetaryValue
quantiles <- quantile(customer_rfm$Recency, probs = c(0.25, 0.5, 0.75))
quantiles_dict <- list(
  Recency = quantile(customer_rfm$Recency, probs = c(0.25, 0.5, 0.75)),
  Frequency = quantile(customer_rfm$Frequency, probs = c(0.25, 0.5, 0.75)),
  MonetaryValue = quantile(customer_rfm$MonetaryValue, probs = c(0.25, 0.5, 0.75))
)

```

```

# Define scoring functions
RScore <- function(x, p, d) {
  if (x <= d[[p]][1]) {
    return(4)
  } else if (x <= d[[p]][2]) {
    return(3)
  } else if (x <= d[[p]][3]) {
    return(2)
  } else {
    return(1)
  }
}

FMScore <- function(x, p, d) {
  if (x <= d[[p]][1]) {
    return(1)
  } else if (x <= d[[p]][2]) {
    return(2)
  } else if (x <= d[[p]][3]) {
    return(3)
  } else {
    return(4)
  }
}

# Assign RFM scores
customer_rfm$R <- sapply(customer_rfm$Recency, RScore, p = "Recency", d = quantiles_dict)
customer_rfm$F <- sapply(customer_rfm$Frequency, FMScore, p = "Frequency", d = quantiles_dict)
customer_rfm$M <- sapply(customer_rfm$MonetaryValue, FMScore, p = "MonetaryValue", d = quantiles_dict)

# Combine RFM scores into a single RFM group and score
customer_rfm$RFMGroup <- paste0(customer_rfm$R, customer_rfm$F, customer_rfm$M)
customer_rfm$RFMScore <- rowSums(customer_rfm[, c("R", "F", "M")])

# Prepare data for clustering
rfm_data <- customer_rfm[, c("Recency", "Frequency", "MonetaryValue")]
scaled_rfm_data <- scale(rfm_data) # Scale the data

```

Elbow Method for Optimal Clusters

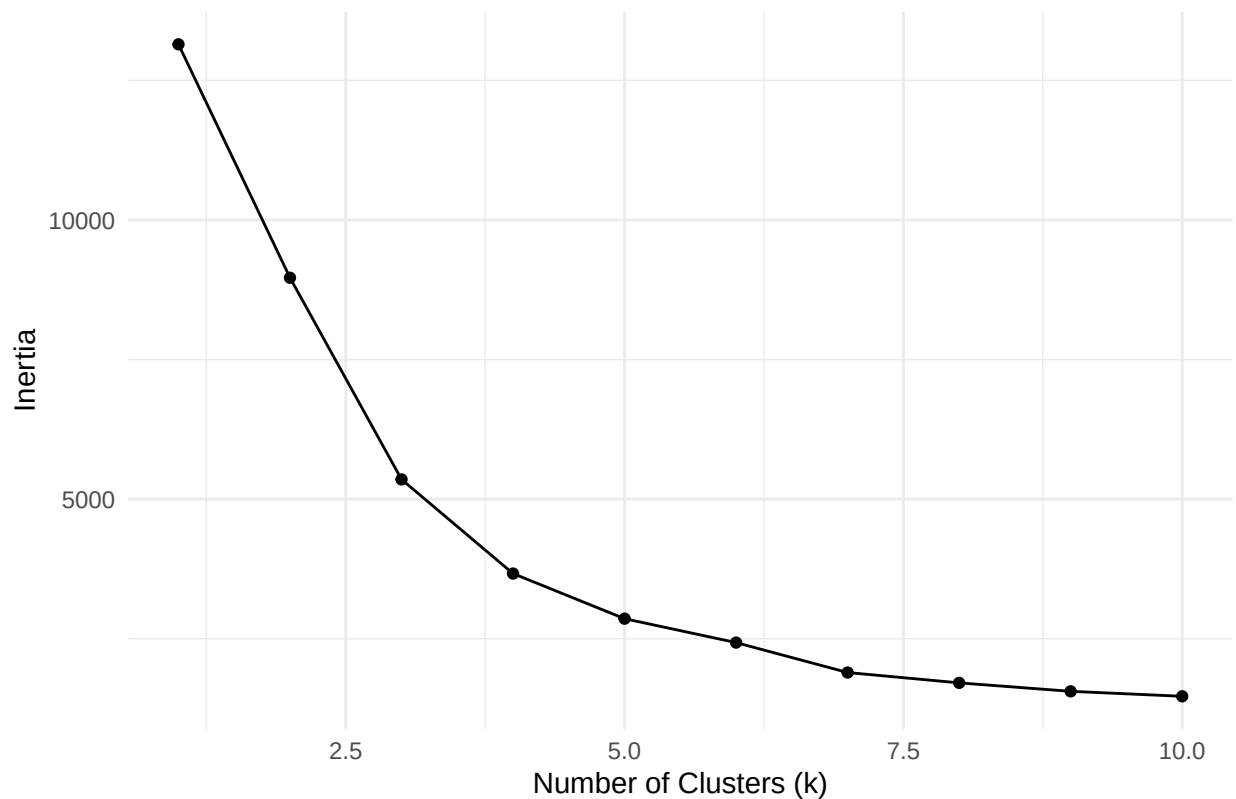
```

# Calculate inertia for different values of k
inertia <- numeric(10)
for (i in 1:10) {
  kmeans_result <- kmeans(scaled_rfm_data, centers = i, nstart = 25)
  inertia[i] <- kmeans_result$tot.withinss
}

# Plot the elbow method
ggplot(data.frame(k = 1:10, inertia = inertia), aes(x = k, y = inertia)) +
  geom_line() +
  geom_point() +
  labs(title = "Elbow Method for Optimal k", x = "Number of Clusters (k)", y = "Inertia") +
  theme_minimal()

```

Elbow Method for Optimal k



K-Means Clustering

```
set.seed(42) # For reproducibility
kmeans_result <- kmeans(scaled_rfm_data, centers = 3, nstart = 25)
customer_rfm$Cluster <- kmeans_result$cluster
```

Cluster Analysis

```
# Summarize cluster characteristics
cluster_analysis <- customer_rfm %>%
  group_by(Cluster) %>%
  summarise(
    Recency_mean = mean(Recency),
    Recency_median = median(Recency),
    Recency_min = min(Recency),
    Recency_max = max(Recency),
    Frequency_mean = mean(Frequency),
    Frequency_median = median(Frequency),
    Frequency_min = min(Frequency),
    Frequency_max = max(Frequency),
    MonetaryValue_mean = mean(MonetaryValue),
    MonetaryValue_median = median(MonetaryValue),
    MonetaryValue_min = min(MonetaryValue),
    MonetaryValue_max = max(MonetaryValue),
    .groups = 'drop'
  )
```

```
print(cluster_analysis)
```

```
## # A tibble: 3 x 13
##   Cluster Recency_mean Recency_median Recency_min Recency_max Frequency_mean
##   <int>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1     1         243.         233.        135.        373.         1.90
## 2     2          40.7          30.3          0         146.         5.96
## 3     3           4.15          2.30         0.107        14.4         118.
## # i 7 more variables: Frequency_median <dbl>, Frequency_min <int>,
## #   Frequency_max <int>, MonetaryValue_mean <dbl>, MonetaryValue_median <dbl>,
## #   MonetaryValue_min <dbl>, MonetaryValue_max <dbl>
```

Visualize Clusters

```
# Recency Distribution by Cluster
plot1 <- ggplot(customer_rfm, aes(x = factor(Cluster), y = Recency)) +
  geom_boxplot() +
  labs(title = "Recency Distribution by Cluster", x = "Cluster", y = "Recency")

# Frequency Distribution by Cluster
plot2 <- ggplot(customer_rfm, aes(x = factor(Cluster), y = Frequency)) +
  geom_boxplot() +
  labs(title = "Frequency Distribution by Cluster", x = "Cluster", y = "Frequency")

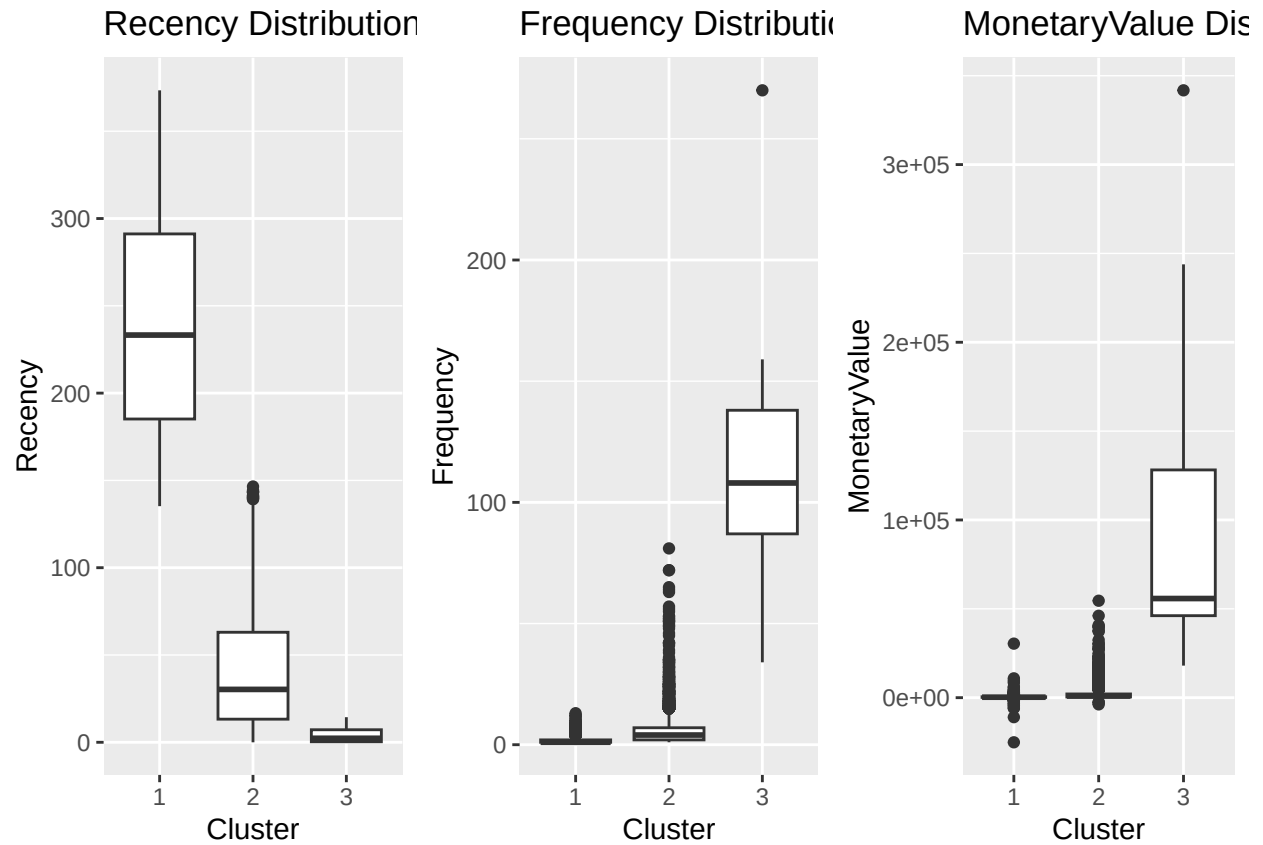
# MonetaryValue Distribution by Cluster
plot3 <- ggplot(customer_rfm, aes(x = factor(Cluster), y = MonetaryValue)) +
  geom_boxplot() +
  labs(title = "MonetaryValue Distribution by Cluster", x = "Cluster", y = "MonetaryValue")

# Combine plots
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'

## The following object is masked from 'package:dplyr':
##
##   combine
```

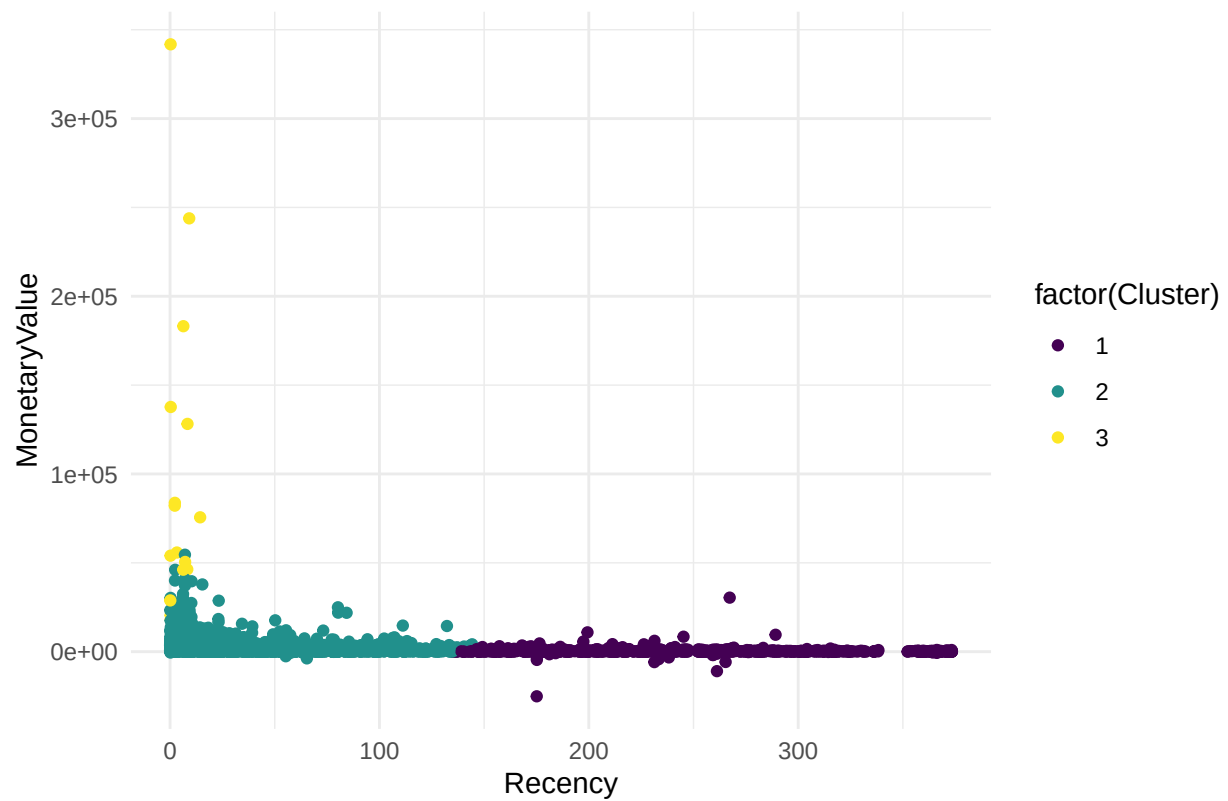
```
grid.arrange(plot1, plot2, plot3, ncol = 3)
```



Scatterplots for Clusters

```
# Recency vs MonetaryValue
ggplot(customer_rfm, aes(x = Recency, y = MonetaryValue, color = factor(Cluster))) +
  geom_point() +
  scale_color_viridis_d() +
  labs(title = "Customer Segmentation by Recency and MonetaryValue", x = "Recency", y = "MonetaryValue")
  theme_minimal()
```

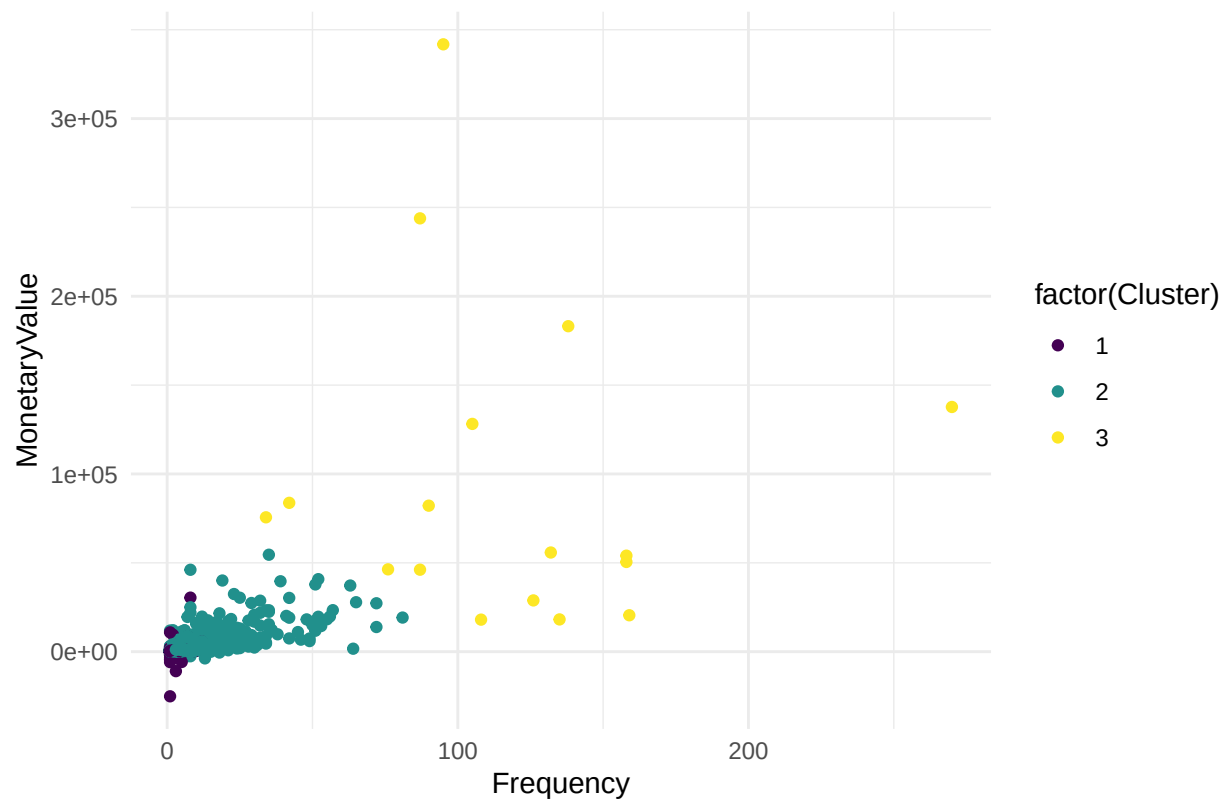
Customer Segmentation by Recency and MonetaryValue



```
# Frequency vs MonetaryValue
```

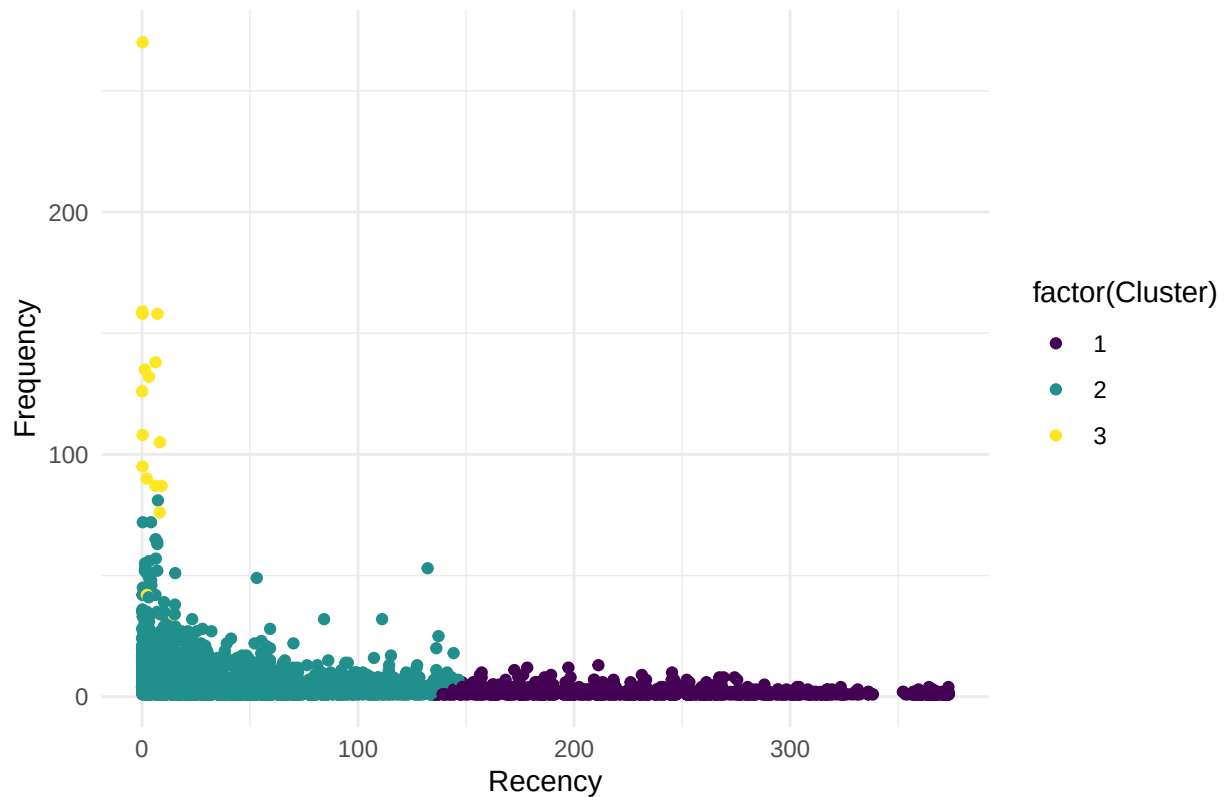
```
ggplot(customer_rfm, aes(x = Frequency, y = MonetaryValue, color = factor(Cluster))) +  
  geom_point() +  
  scale_color_viridis_d() +  
  labs(title = "Customer Segmentation by Frequency and MonetaryValue", x = "Frequency", y = "MonetaryValue") +  
  theme_minimal()
```


Customer Segmentation by Frequency and MonetaryValue



```
# Recency vs Frequency  
ggplot(customer_rfm, aes(x = Recency, y = Frequency, color = factor(Cluster))) +  
  geom_point() +  
  scale_color_viridis_d() +  
  labs(title = "Customer Segmentation by Recency and Frequency", x = "Recency", y = "Frequency") +  
  theme_minimal()
```

Customer Segmentation by Recency and Frequency



Cluster Distribution

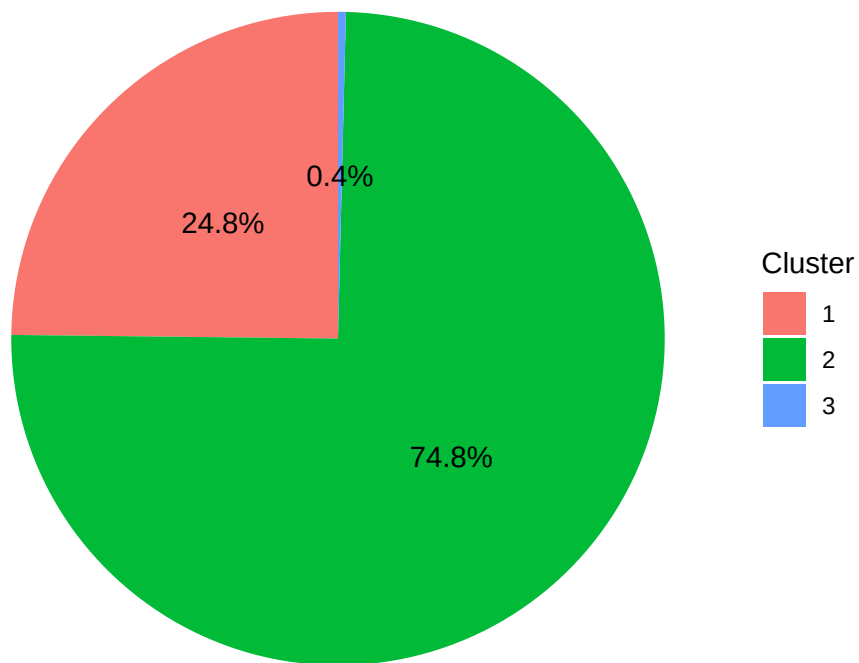
```
# Count the number of customers in each cluster
cluster_counts <- table(customer_rfm$Cluster)

# Create a data frame for the pie chart
pie_data <- data.frame(Cluster = names(cluster_counts), Count = as.numeric(cluster_counts))

# Calculate percentages
pie_data$Percentage <- pie_data$Count / sum(pie_data$Count) * 100

# Pie chart
ggplot(pie_data, aes(x = "", y = Count, fill = Cluster)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar("y", start = 0) +
  geom_text(aes(label = paste0(round(Percentage, 1), "%")),
            position = position_stack(vjust = 0.5), size = 4) +
  labs(title = "Distribution of Customer Clusters", fill = "Cluster") +
  theme_void() +
  theme(legend.position = "right")
```

Distribution of Customer Clusters



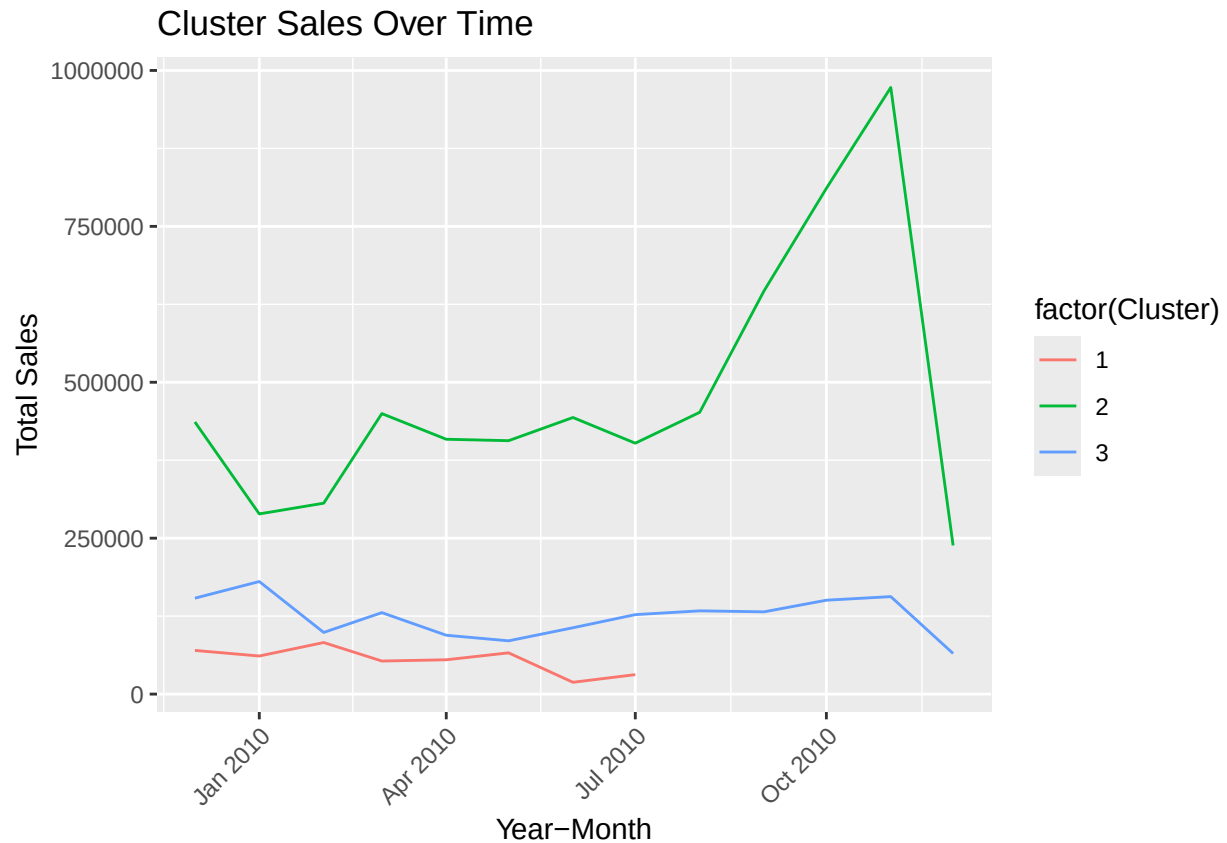
Sales Trends by Cluster

```
# Merge and group by Cluster and YearMonth
cluster_sales_over_time <- df_no_duplicates %>%
  left_join(customer_rfm, by = "CustomerID") %>%
  group_by(Cluster, YearMonth) %>%
  summarise(Sales = sum(Sales), .groups = 'drop')

# Convert YearMonth to timestamp
cluster_sales_over_time$YearMonth <- as.Date(paste0(cluster_sales_over_time$YearMonth, "-01"))

# Filter the data to only include 3 clusters
cluster_sales_over_time_filtered <- cluster_sales_over_time %>%
  filter(Cluster %in% c(1, 2, 3))

# Plot the sales trends for only 3 clusters
ggplot(cluster_sales_over_time_filtered, aes(x = YearMonth, y = Sales, color = factor(Cluster))) +
  geom_line() +
  labs(title = "Cluster Sales Over Time", x = "Year-Month", y = "Total Sales") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

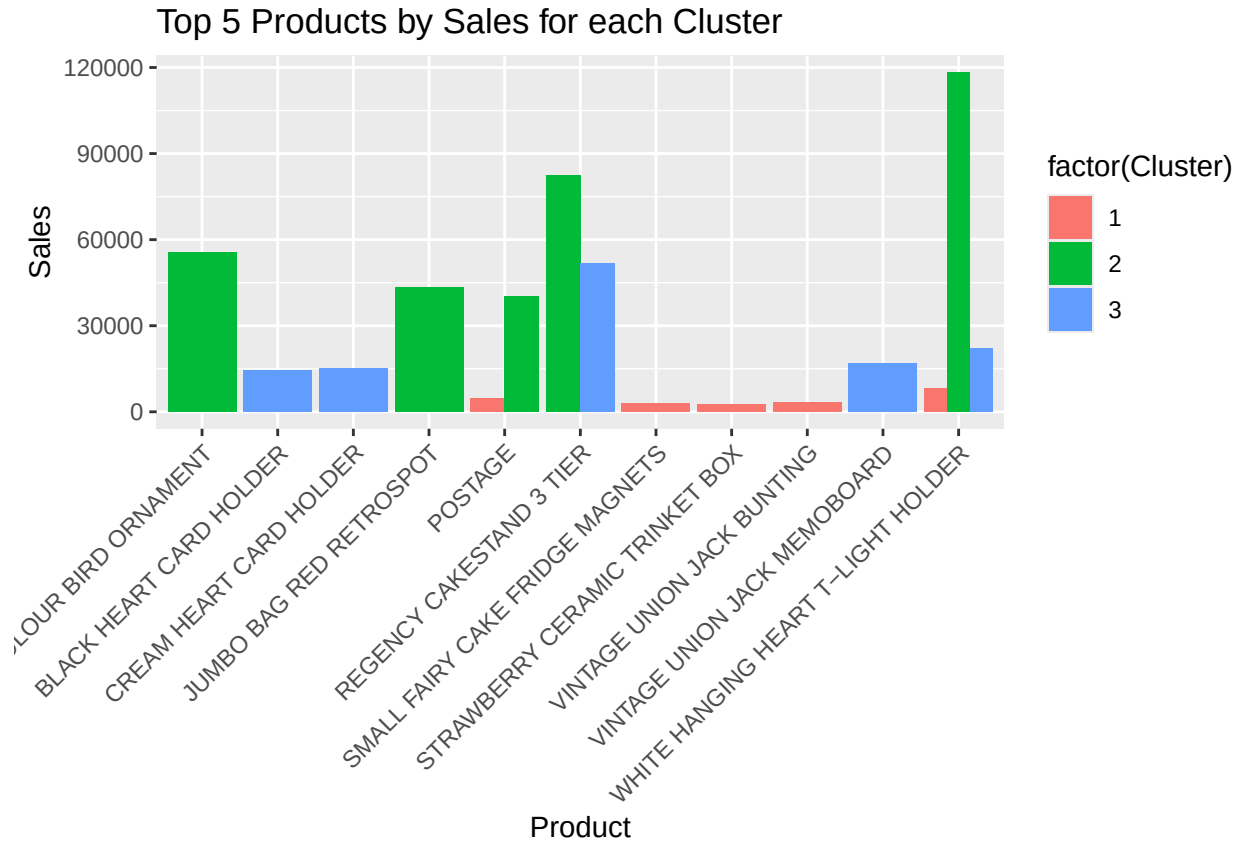


Top Products by Cluster

```
# Group by Cluster and Description, calculate total sales
top_products_by_cluster <- df_no_duplicates %>%
  left_join(customer_rfm, by = "CustomerID") %>%
  group_by(Cluster, Description) %>%
  summarise(Sales = sum(Sales), .groups = "drop") %>%
  group_by(Cluster) %>%
  arrange(Cluster, desc(Sales)) %>%
  slice_head(n = 5) %>%
  ungroup()

# Filter the data to only include 3 clusters
top_products_by_cluster <- top_products_by_cluster %>%
  filter(Cluster %in% c(1, 2, 3))

# Plotting the top products per cluster
ggplot(top_products_by_cluster, aes(x = Description, y = Sales, fill = factor(Cluster))) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Top 5 Products by Sales for each Cluster", x = "Product", y = "Sales") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



There are three clusters. Cluster 2 has the highest monetary value with the shortest recency, but the least frequency. Cluster 1 has the lowest monetary value, the longest recency, and an average frequency of 1, indicating only one purchase. Interestingly, this cluster ceased purchasing in July 2020. Further investigation is needed to understand the reasons behind this.

The high-value Cluster 2 maintains a stable purchasing pattern through the year, while Cluster 1 shows a seasonal increase starting in August 2010, peaking in November 2010. Although their average purchase amount is not the highest, their total spend accounts for the majority of sales.

We also found that the three clusters have different product preferences, which highlights the potential for targeted marketing.